

FDE Zero-to-Expert Roadmap

A focused learning plan for GenAI Engineering, RAG, Agentic AI and Forward Deployed Engineering

Rule: follow one primary resource per topic, build after every topic, and avoid framework hopping.

Goal: become capable of designing, building, debugging, evaluating and deploying real enterprise GenAI systems - not just completing courses.

1. Core Learning Roadmap

#	Topic	Primary resource	Type	Outcome
1	How LLMs actually work	DeepLearning.AI - How Transformer LLMs Work	Video + code	Explain tokens, embeddings, attention, transformers, context windows and inference.
2	Prompt Engineering	DeepLearning.AI - ChatGPT Prompt Engineering for Developers	Video + code	Write reliable prompts, delimit context, extract structured information and improve outputs iteratively.
3	LLM Application Basics	DeepLearning.AI - Building Systems with the ChatGPT API	Video + code	Build multi-step LLM workflows and validate intermediate and final outputs.
4	Embeddings + Vector Databases	DeepLearning.AI - Vector Databases: From Embeddings to Applications	Video + labs	Create embeddings, understand similarity search, and store/retrieve vectors.
5	RAG - properly	DeepLearning.AI - Retrieval Augmented Generation	Full course	Build strong RAG systems: chunking, semantic/hybrid retrieval, vector databases, reranking and evaluation.
6	LangChain Basics	DeepLearning.AI - LangChain for LLM Application Development	Video + code	Understand chains, retrievers, memory and application composition.
7	Tool / Function Calling	DeepLearning.AI - Functions, Tools and Agents with LangChain	Video + code	Make an LLM choose and invoke tools/APIs instead of only generating text.
8	LangGraph / Orchestration	LangChain Academy - Introduction to LangGraph	Course + labs	Build stateful graphs with routing, loops, memory, human approval and agents.
9	AI Agents	Hugging Face - AI Agents Course	Text + practical	Understand the agent loop, planning, tools, observations, Agentic RAG and multi-step agents.
10	MCP	Hugging Face - Model Context Protocol Course	Text + practical	Build MCP clients/servers and expose enterprise tools/resources to AI systems.
11	Evals + Observability	DeepLearning.AI - Evaluating and Debugging Generative AI	Video + code	Trace runs, define evals, compare versions and detect regressions.
12	Guardrails / Safety	DeepLearning.AI - Quality and Safety for LLM Applications	Video + code	Handle hallucination, jailbreaks, leakage and quality/safety checks in production.

2. How to Study

Do not watch everything first and build later. Use this loop for every topic:

Learn topic -> build tiny example yourself -> add the same capability to the IBM i AI project -> break it -> debug it -> move on

Embeddings

Learning exercise: Embed 100 text paragraphs.

Your project: Embed RPG programs/procedures and find semantically similar code.

RAG

Learning exercise: Ask questions over a document set.

Your project: Ask: "Where is customer credit limit validated?" Retrieve RPG + called procedures + schema/docs, then answer with evidence.

Tool Calling

Learning exercise: Let a model call a simple calculator or API.

Your project: Expose a tool like `get_program_references("ORDENTR")` backed by IBM i metadata.

MCP

Learning exercise: Build a small MCP client/server.

Your project: Expose IBM i tools/resources such as object info, program references, file definitions, job info and safe SQL.

LangGraph

Learning exercise: Build a graph with state and conditional routing.

Your project: Create Analyze -> Dependencies -> Retrieve Context -> Explain Business Logic -> Human Review -> Modernization Plan -> Tests.

3. Your Portfolio Project

Build one serious system throughout the roadmap instead of twenty disconnected demos:

```
IBM i AI Modernization Platform
- RPG / source ingestion
- Dependency analyzer
- Embedding + retrieval layer
- RAG documentation assistant
- Business-rule extraction
- Tool calling
- LangGraph modernization workflow
- Agents for analysis / conversion / review
- MCP integration
- Test generation + evaluation
- Observability + guardrails
- Simple UI / action board
```

4. 13-Week Timeline

Time	Focus
Week 1	LLM Fundamentals + Prompt Engineering + Building LLM Systems
Week 2	Embeddings + Vector Databases
Weeks 3-5	RAG - spend real time here
Week 6	LangChain + Tool Calling
Weeks 7-8	LangGraph / orchestration
Weeks 9-10	Agents
Week 11	MCP
Week 12	Evals + Observability + Guardrails
Week 13 onward	Productionize the IBM i AI Modernization Platform

5. Framework Rule

Do not study CrewAI, AutoGen, LlamaIndex, Semantic Kernel, ADK and every other framework right now.

First become strong at the concepts:

LLM -> retrieval -> tools -> state -> agents -> orchestration -> MCP -> evals -> production

Once these concepts are solid, a new framework is mostly an API-learning problem rather than a conceptual-learning problem.

6. Language Strategy

Use Python when a course/lab requires it, but build your own portfolio project primarily in TypeScript/Node.js where practical. Keep your existing backend advantage while becoming comfortable enough with Python to work in the broader AI ecosystem.

7. Completion Standard

Do not mark a topic complete because you watched the course. Mark it complete only when you can:

- Explain the concept in simple language without notes.
- Implement a small version yourself.
- Apply it to the IBM i project.
- Debug at least one failure mode.
- Explain when NOT to use it.
- Defend your architecture choices in an interview/client discussion.

Primary objective: become an engineer who can solve enterprise problems with GenAI - not a person who only knows RAG/agent terminology.